ECE131a Extra Credit Project
Distributed Detection in Gaussian Noise

Wesley Hon

June 5, 2026

Contents

1 Introduction

1.1 Background

This problem is adapted from the results in [1] and [2]. A scalar signal $S \in \{-m, m\}$ is transmitted from a satellite to a field of n sensors on the surface of the earth. S communicates an important message, and is equally likely to take either value so that

$$P_S(s) = \begin{cases} \frac{1}{2}, & \text{for } s = -m, \\ \frac{1}{2}, & \text{for } s = m, \\ 0, & \text{otherwise.} \end{cases}$$

Sensor i observes random variable

$$X_i = S + Z_i$$

where the random variables Z_i are independent and identically distributed according to a standard normal distribution $\mathcal{N}(0, 1)$. This project explores how to efficiently and reliably estimate S from the vector of observations

$$\mathbf{X} = [X_1, X_2, \ldots, X_n].$$

Any estimate $\hat{S}$ will naturally depend on the observables. A *statistic* is a function of the set of observables that we use in preparing our estimate. For this problem we will be especially interested in the statistic

$$T_n = \frac{1}{n} \sum_{i=1}^{n} X_i,$$

which is sometimes called the sample mean.
A *sufficient statistic* of a set of observations for estimation of a particular random variable is one that allows as good of an estimate of the random variable as the original observations themselves. T_n is a sufficient statistic of $\mathbf{X}$ for S if and only if

$$f_{X|T,S}(x|t, s) = f_{X|T}(x|t).$$

In our specific case, this turns out to be true so that the sample mean is a sufficient statistic of $\mathbf{X}$ for S. We can use the sample mean T_n to estimate S and be confident that we have not lost any information. Specifically, we will estimate S as follows:

$$\hat{S} = \begin{cases} m, & \text{if } T_n \geq 0, \\ -m, & \text{otherwise.} \end{cases}$$

1.2 Report Organization

The project is 25 project points which is equivalent to 5% of the total grade percentage as extra credit. The project must be completed to earn an A+ grade.
There are 8 problems, each answer must have a Matlab plot to back up the answer.

Project Statement:
ECE 131a Spring 2026 Extra Credit Project, Distributed Detection in Gaussian Noise

2 Problem 1

3pts.

Suppose that $m = 0.5$. Use Matlab function `semilogy` to plot the probability of error $P(\hat{S} \neq S)$ as a function of the number of sensors n for $1 \leq n \leq 100$. The Matlab function `qfunc` is your friend for this problem. How many sensors do you need so that $P(\hat{S} \neq S) < 10^{-3}$? Call this number n_g (as in the number of Gaussian samples).

For this problem, each sensor observes

$$X_i = S + Z_i$$

where $S \in \{-m, m\}$ and the noise variables Z_i are independent and identically distributed according to a standard normal distribution $N(0, 1)$. The signal amplitude is given as $m = 0.5$.

The detector uses the sample mean

$$T_n = \frac{1}{n} \sum_{i=1}^{n} X_i$$

and makes the decision

$$\hat{S} = \begin{cases} m, & T_n \geq 0 \\ -m, & T_n < 0 \end{cases}$$

The goal is to determine the probability of error as a function of the number of sensors n and find the minimum number of sensors required so that the probability of error is less than 10^{-3}.

2.1 Distribution of the Sample Mean

Starting with the definition of the sample mean,

$$T_n = \frac{1}{n} \sum_{i=1}^{n} (S + Z_i)$$

Since S is constant across all sensors,

$$T_n = S + \frac{1}{n} \sum_{i=1}^{n} Z_i$$

Because the noise variables Z_i are independent Gaussian random variables with mean 0 and variance 1, the average noise term has distribution

$$\frac{1}{n} \sum_{i=1}^{n} Z_i \sim N\left(0, \frac{1}{n}\right)$$

Therefore, conditioned on the transmitted signal S,

$$T_n|S = m \sim N\left(m, \frac{1}{n}\right)$$

and

$$T_n|S = -m \sim N\left(-m, \frac{1}{n}\right)$$

Thus, the sample mean remains to be Gaussian with mean equal to the transmitted signal (m) and variance equal to $1/n$.

2.2 Probability of Error

An error occurs when the detector chooses the wrong hypothesis.
Considering the case $S = m$, the detector makes an error whenever $T_n < 0$. Therefore,

$$P(\text{error}|S = m) = P(T_n < 0|S = m)$$

Since $T_n|S = m$ is Gaussian with mean m and variance $1/n$, we are able to standardize the random variable:

$$P(\text{error}|S = m) = P\left(\frac{T_n - m}{1/\sqrt{n}} < \frac{0 - m}{1/\sqrt{n}}\right)$$

$$= P(Z < -m\sqrt{n})$$

where $Z \sim N(0, 1)$.
Using the Gaussian Q-function,

$$P(\text{error}|S = m) = Q(m\sqrt{n})$$

By symmetry,

$$P(\text{error}|S = -m) = Q(m\sqrt{n})$$

Since both of the hypotheses are equally likely,

$$P(\text{error}) = \frac{1}{2}P(\text{error}|S = m) + \frac{1}{2}P(\text{error}|S = -m)$$

$$= Q(m\sqrt{n})$$

Substituting $m = 0.5$:

$$P(\text{error}) = Q(0.5\sqrt{n})$$

This expression above is the theoretical probability of error for any number of sensors n.

2.3 MATLAB

Listing 1: Matlab Code: Problem 1

```
clc;
clear;
close all;

%% Problem 1
% Distributed Detection in Gaussian Noise
% P(error) = Q(m*sqrt(n))

m = 0.5;                % Signal amplitude
n = 1:100;              % Number of sensors

% Compute probability of error, using qfunc
Pe = qfunc(m*sqrt(n));

% Error threshold (10^-3)
threshold = 1e-3;

% Find smallest n satisfying Pe < 10^-3
ng = find(Pe < threshold, 1, 'first');

%% Create Figure
figure;

semilogy(n, Pe, 'LineWidth', 2); % using semilogy
hold on;
grid on;

% Threshold line
yline(threshold,'--','10^{-3}','LineWidth',1.5);

% Mark ng
semilogy(ng, Pe(ng), 'o', ...
    'MarkerSize', 8, ...
    'LineWidth', 2);

xlabel('Number of Sensors, n');
ylabel('Probability of Error');
title('Probability of Error vs. Number of Sensors (m = 0.5)');

legend('P_e = Q(0.5\sqrt{n})', ...
       'Threshold', ...
       sprintf('n_g = %d',ng), ...
       'Location','southwest');

%% Displaying Results (print)
fprintf('\n');
fprintf('-----------------------------------\n');
fprintf('Problem 1 Results\n');
```

```
fprintf('-----------------------------------\n');
fprintf('Signal amplitude m = %.1f\n',m);
fprintf('Required error probability < 10^-3\n');
fprintf('Minimum number of sensors n_g = %d\n',ng);
fprintf('Probability of error at n_g = %.6e\n',Pe(ng));

if ng > 1
    fprintf('Probability of error at n = %d is %.6e\n', ...
        ng-1, Pe(ng-1));
end
fprintf('-----------------------------------\n');
```

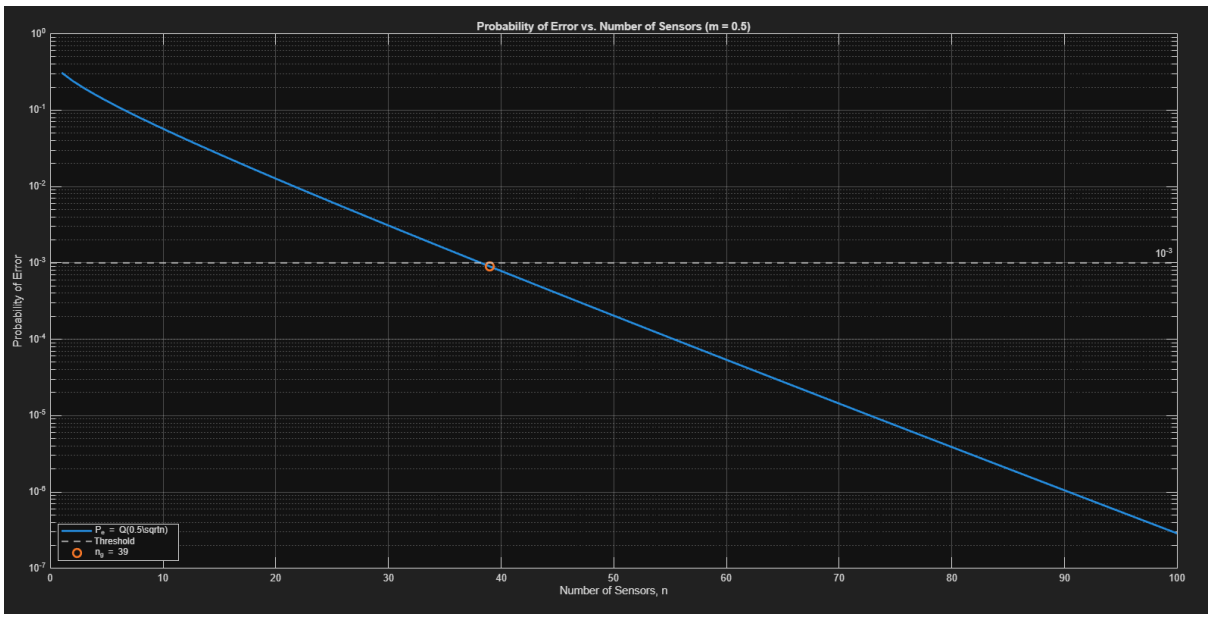

Figure 1: Probability of error as a function of the number of sensors for $m = 0.5$.

```
-----------------------------------
Problem 1 Results
-----------------------------------
Signal amplitude m = 0.5
Required error probability < 10^-3
Minimum number of sensors n_g = 39
Probability of error at n_g = 8.966136e-04
Probability of error at n = 38 is 1.027359e-03
-----------------------------------
```

Figure 2: MATLAB command window output for Problem 1.

The theoretical probability of error derived in the previous section is

$$P(\hat{S} \neq S) = Q(0.5\sqrt{n}).$$

This expression was evaluated in MATLAB for values of n (the number of sensors) ranging from 1 to 100. The built-in `qfunc` function was used to compute the Gaussian Q-function, and the results were plotted using the `semilogy` command. A logarithmic scale was used for the vertical axis because the probability of error decreases rapidly as the number of sensors increases.

Figure 1 shows the probability of error as a function of the number of sensors. The graph shows that the probability of error decreases steadily as n increases in the logarithmic scale. This makes sense because averaging more independent sensor measurements reduces the variance of the sample mean, making it easier to distinguish between the two possible transmitted signal values, thereby ensuring our confidence in an accurate signal.

A horizontal reference line at 10^{-3} was added to the plot to show the required error threshold. From the graph, the error probability crosses this threshold between $n = 38$ and $n = 39$. MATLAB was then used to find the smallest value of n satisfying

$$P(\hat{S} \neq S) < 10^{-3}.$$

The numerical results show that

$$P_e(38) = 1.027359 \times 10^{-3}$$

and

$$P_e(39) = 8.966136 \times 10^{-4}.$$

Therefore, $n = 38$ is not enough, since its error probability is still above 10^{-3}. However, $n = 39$ is sufficient because its error probability is below 10^{-3}. Thus, the minimum number of sensors required is

$$\boxed{n_g = 39}$$

3 Problem 2

4pts.

Simulate this sensor system with the n_g sensors as follows: For each simulation run, generate n_g observations X_i using $S = 0.5$ and `normrnd` to generate Z_i and use them to compute the sufficient statistic T_{n_g}. Then use T_{n_g} to compute $\hat{S}$ and note whether the estimate is correct or is an error. Perform simulation runs until you have observed 200 errors. Keep track of the total number K of simulation runs. Report your estimated error rate as $200/K$. If everything is working well, your simulation point should match the probability of error you calculated in the previous part for n_g observations, i.e., a number close to 10^{-3}. Your simulation may take a few minutes to complete since you will need approximately 200,000 runs.

3.1 Preliminary Remarks

In this problem, we should verify the result calculated in Problem 1 using a Matlab simulation. It was figured out that the minimum number of sensors required to make the probability of error less than 10^{-3} was found to be

$$n_g = 39.$$

The transmitted signal is fixed as

$$S = 0.5.$$

Each sensor observes

$$X_i = S + Z_i,$$

where

$$Z_i \sim N(0, 1).$$

For each simulation run, $n_g = 39$ independent noise samples are generated, producing 39 sensor observations. The sample mean is then computed as

$$T_{n_g} = \frac{1}{n_g} \sum_{i=1}^{n_g} X_i.$$

The detector uses the rule

$$\hat{S} = \begin{cases} 0.5, & T_{n_g} \geq 0, \\ -0.5, & T_{n_g} < 0. \end{cases}$$

Since the true value is $S = 0.5$, an error occurs whenever the detector decides $\hat{S} = -0.5$. There is a deviation. This happens when the sample mean is negative:

$$T_{n_g} < 0.$$

The simulation is repeated until 200 errors are observed. If K is the total number of simulation runs required to observe those 200 errors, then the simulated probability of error is estimated as

$$\hat{P}_e = \frac{200}{K}.$$

This estimate should be close to the theoretical value $P_e(39)$.

$$P_e(39) = Q(0.5\sqrt{39}) \approx 8.966136 \times 10^{-4}.$$

Because the error probability is close to 10^{-3}, approximately

$$K \approx \frac{200}{8.966136 \times 10^{-4}}$$

simulation runs are expected. This gives

$$\boxed{K \approx 223{,}061}$$

Therefore, the simulation should require around $220,000$ runs before reaching 200 errors. The exact value of K may vary each time the simulation is run because the Gaussian noise samples are random. However, if the simulation is correct, the estimated probability of error $200/K$ should be close to 10^{-3}, confirming the theoretical result that I calculated in Problem 1.

3.2 Matlab Code

Listing 2: Matlab Code: Problem 2

```
clc;
clear;
close all;

%% Problem 2
% Simulation of full-precision sensor system

m = 0.5;                    % Signal amplitude
S = 0.5;                    % Transmitted signal for simulation
ng = 39;                    % Number of sensors calculated from p1

target_errors = 200;        % Stop simulation after 200 errors
num_errors = 0;             % Error counter
K = 0;                      % Total simulation runs

%% Simulation Loop

while num_errors < target_errors

    % Increase simulation run counter
    K = K + 1;

    % Generate ng independent Gaussian noise samples
```

```
    Z = normrnd(0, 1, [1, ng]);

    % Generate sensor observations
    X = S + Z;

    % Compute sufficient statistic
    T = mean(X);

    % Decision rule
    if T >= 0
        S_hat = 0.5;
    else
        S_hat = -0.5;
    end

    % Check if an error occurred
    if S_hat ~= S
        num_errors = num_errors + 1;
    end
end

%% Compute Error Probability

Pe_sim = target_errors / K;
Pe_theory = qfunc(m * sqrt(ng));

%% Display Results

fprintf('\n');
fprintf('-----------------------------------\n');
fprintf('Problem 2 Simulation Results\n');
fprintf('-----------------------------------\n');
fprintf('Number of sensors n_g = %d\n', ng);
fprintf('Target number of errors = %d\n', target_errors);
fprintf('Total number of simulation runs K = %d\n', K);
fprintf('Estimated error probability = 200/K = %.6e\n', Pe_sim);
fprintf('Theoretical error probability = %.6e\n', Pe_theory);
fprintf('-----------------------------------\n');
```

```
-----------------------------------
Problem 2 Simulation Results
-----------------------------------
Number of sensors n_g = 39
Target number of errors = 200
Total number of simulation runs K = 220803
Estimated error probability = 200/K = 9.057848e-04
Theoretical error probability = 8.966136e-04
-----------------------------------
```

Figure 3: Problem 2 MATLAB simulation output (Run 1).

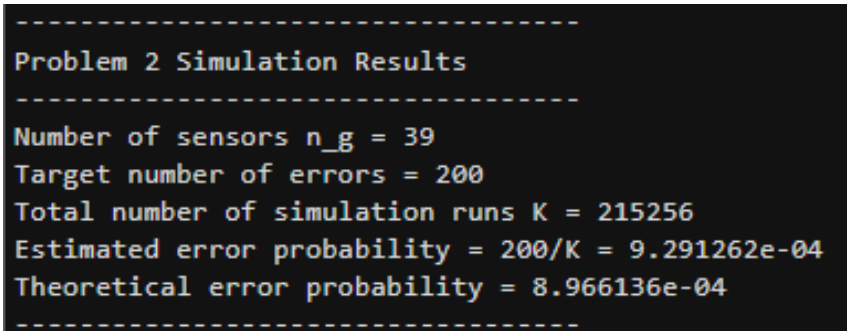

Figure 4: Problem 2 MATLAB simulation output (Run 2).

3.3 Interpretation of Results

The MATLAB simulation was performed twice using the full-precision sensor system with $n_g = 39$ sensors. In each trial, the simulation continued until 200 detection errors were observed. The probability of error was then estimated using

$$\hat{P}_e = \frac{200}{K},$$

where K is the total number of simulation runs required to accumulate 200 errors.

For Run 1, the simulation required

$$K = 220{,}803$$

trials, resulting in an estimated probability of error of

$$\hat{P}_{e,1} = 9.057848 \times 10^{-4}.$$

For Run 2, the simulation required

$$K = 215{,}256$$

trials, resulting in an estimated probability of error of

$$\hat{P}_{e,2} = 9.291262 \times 10^{-4}.$$

These values are very close to the theoretical probability of error obtained earlier,

$$P_e(39) = 8.966136 \times 10^{-4}.$$

The small differences between the simulated and theoretical values are expected because the simulation relies on randomly generated Gaussian noise samples. Each simulation run produces a different outcome of the noise process, causing slight variations in the estimated error probability. Despite these random fluctuations, both simulation results are within a few percent of the theoretical prediction.

The results therefore confirm the analysis from Problem 1. Both simulation trials produced error probabilities on the order of 10^{-3}, demonstrating that 39 sensors are sufficient to achieve the desired level of reliability. The close agreement between the theoretical and simulated probabilities of error verifies that the sample mean detector does accurately model the performance of the full-precision sensor system.

4 Problem 3

3pts.

We will refer to the sensor studied in problems 1–2 as a "full precision" sensor. Now, let's suppose that it is resource intensive to sense the real numbers X_i with full precision. We want to deploy less expensive sensors that merely detect whether $X_i \geq 0$. These sensors each collect

$$B_i = \begin{cases} 1 & \text{if } X_i \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

We are fortunate that $U_n = \sum_{i=1}^{n} B_i$ is a sufficient statistic of B for S, so that we can base our estimate $\hat{S}$ on this sum.
What kind of random variable is B_i? What kind of random variable is U_n? For our case of interest, where $S = \pm 0.5$ and $X_i = S + Z_i$ with the random variables Z_i independent and identically distributed according to a standard normal distribution $\mathcal{N}(0, 1)$, find the value of $P(B_i = 1 \mid S = 0.5)$, which we will call p.
We need a decision rule to decide our estimate $\hat{S}$ as a function of U_n. We will use the "maximum likelihood" principle to choose our rule. Specifically, choose the value of $\hat{S}$ the rule that maximizes the probability of the observed value of U_n given $S = \hat{S}$.

$$\hat{S} = \begin{cases} 0.5 & \text{if } P_{U|S}(u|0.5) > P_{U|S}(u|-0.5) \\ -0.5 & \text{otherwise.} \end{cases}$$

Show that the maximum-likelihood decoder turns out to be a majority vote decoder. It turns out that how you handle ties does not affect the total probability of error, but it could favor one of the outcomes.

4.1 part A

4.1.1 Preliminary Remarks

In Problems 1 and 2, each sensor measured the full-precision observation

$$X_i = S + Z_i,$$

where $S \in \{-0.5, 0.5\}$ and $Z_i \sim N(0, 1)$. In Problem 3, the sensors are simplified so that they no longer record the full value of X_i. Instead, each sensor records only whether its observation is positive or negative.

The binary sensor output is defined as

$$B_i = \begin{cases} 1, & X_i \geq 0, \\ 0, & X_i < 0. \end{cases}$$

The sufficient statistic for the collection of binary observations is

$$U_n = \sum_{i=1}^{n} B_i,$$

which counts the total number of sensors that report a value of 1. The goal of this problem is to identify the distributions of B_i and U_n. Then, determine the value of

$$p = P(B_i = 1|S = 0.5).$$

4.1.2 Answer

Since B_i can only take the values 0 and 1, it is a Bernoulli random variable.

Because U_n is the sum of n independent Bernoulli random variables, U_n follows a binomial distribution.

For $S = 0.5$,

$$X_i = 0.5 + Z_i.$$

The probability that a sensor reports $B_i = 1$ is

$$p = P(B_i = 1|S = 0.5).$$
$$p = P(X_i \geq 0|S = 0.5).$$

Substituting $X_i = 0.5 + Z_i$,

$$p = P(0.5 + Z_i \geq 0).$$
$$p = P(Z_i \geq -0.5).$$

By symmetry of the standard normal distribution,

$$P(Z_i \geq -0.5) = P(Z_i \leq 0.5).$$

Therefore,

$$p = \Phi(0.5).$$

Using the standard normal cumulative distribution function,

$$p = \Phi(0.5) \approx 0.6915.$$

Thus,

$$\boxed{p \approx 0.6915}.$$

Therefore,

$$\boxed{B_i|S = 0.5 \sim \text{Bernoulli}(0.6915)}$$

and

$$\boxed{U_n|S = 0.5 \sim \text{Binomial}(n, 0.6915)}$$

Similarly,

$$P(B_i = 1|S = -0.5) = 1 - p \approx 0.3085,$$
$$\boxed{U_n|S = -0.5 \sim \text{Binomial}(n, 1 - p)}$$

4.2 part B

4.2.1 Preliminary Remarks

The problem states that the estimate $\hat{S}$ should be chosen according to the maximum-likelihood principle. Under maximum-likelihood detection, the detector selects the value of S that makes the observed value of the sufficient statistic U_n most likely.

The decision rule is

$$\hat{S} = \begin{cases} 0.5, & P(U_n = u|S = 0.5) > P(U_n = u|S = -0.5), \\ -0.5, & \text{otherwise.} \end{cases}$$

Since Part A showed that U_n follows a binomial distribution under both hypotheses, the likelihood functions can be written explicitly and compared. The goal is to simplify the maximum-likelihood rule and interpret its representation.

4.2.2 Answer

From Part (a),

$$U_n|S = 0.5 \sim \text{Binomial}(n, p)$$
$$U_n|S = -0.5 \sim \text{Binomial}(n, 1-p).$$

Therefore,

$$P(U_n = u|S = 0.5) = \binom{n}{u} p^u (1-p)^{n-u}$$
$$P(U_n = u|S = -0.5) = \binom{n}{u} (1-p)^u p^{n-u}.$$

The maximum-likelihood detector chooses $S = 0.5$ whenever

$$\binom{n}{u} p^u (1-p)^{n-u} > \binom{n}{u} (1-p)^u p^{n-u}.$$
$$p^u (1-p)^{n-u} > (1-p)^u p^{n-u}.$$

Dividing both sides by $(1-p)^{n-u} p^{n-u}$ gives

$$\left(\frac{p}{1-p}\right)^u > \left(\frac{p}{1-p}\right)^{n-u}.$$

Since

$$p = \Phi(0.5) \approx 0.6915 > 0.5,$$

then:

$$\frac{p}{1-p} > 1.$$

Therefore, the inequality is determined by the exponents:

$$u > n - u.$$

$$2u > n.$$

Thus,

$$u > \frac{n}{2}.$$

The resulting decision rule is

$$\hat{S} = \begin{cases} 0.5, & U_n > \frac{n}{2}, \\ -0.5, & U_n \leq \frac{n}{2}. \end{cases}$$

This means the detector chooses $S = 0.5$ whenever a majority of the sensors report $B_i = 1$. Otherwise, it chooses $S = -0.5$.
Therefore,

The maximum-likelihood decoder is a majority-vote decoder.

If n is even, a tie occurs when $U_n = n/2$. The tie-breaking rule may favor one outcome, but it does not affect the overall probability of error when both signal values are equally likely.

5 Problem 4

3pts.

Use the Matlab function `semilogy` to plot the probability of error $P(\hat{S} \neq S)$ as a function of the number of sensors n for $1 \leq n \leq 100$ for the system based on sensors that provide B_i. Add this to your plot from problem 2 so that you have a single plot with two curves. Label your axes. Include a legend. Take pride in your data. For the binary sensor, you only need to plot probability of error for odd values of n, so you don't need to worry about breaking ties. The Matlab function `binocdf` is your friend for this problem. How many sensors, considering only odd numbers, do you need so that $P(\hat{S} \neq S) < 10^{-3}$? Call this number n_b (as in the number of Bernoulli samples in the binomial random variable.)

5.1 Preliminary Remarks

In Problem 3, the full-precision sensor was replaced with a binary sensor that records only whether the observation is positive or negative. Rather than observing the real-valued quantity

$$X_i = S + Z_i,$$

each sensor produces the binary output

$$B_i = \begin{cases} 1, & X_i \geq 0, \\ 0, & X_i < 0. \end{cases}$$

The sufficient statistic for this binary sensor system is

$$U_n = \sum_{i=1}^{n} B_i,$$

which counts the number of sensors reporting a value of 1.

In Problem 3, it was shown that the maximum-likelihood detector reduces to a majority-vote decoder. Therefore, when $S = 0.5$, a detection error occurs whenever the majority of the sensors vote incorrectly. Since the problem specifies that only odd values of n need to be considered, ties are not possible, and the decision rule is straightforward.

The objective of this problem is to determine the probability of error of the binary sensor system as a function of n (the number of sensors) and compare its performance to the full-precision sensor system from Problems 1 and 2. Both error probability curves must be plotted on the same semilogarithmic graph, and the minimum odd number of binary sensors required to achieve an error probability below 10^{-3} will be identified.

5.2 Answer

From Problem 3,

$$p = P(B_i = 1 | S = 0.5) = \Phi(0.5) \approx 0.6915.$$

Since each B_i is a Bernoulli random variable and the sensors operate independently,

$$U_n|S = 0.5 \sim \text{Binomial}(n, p).$$

The majority-vote detector decides $\hat{S} = 0.5$ whenever more than half of the sensors report $B_i = 1$. Because only odd values of n are considered, ties are not possible.

When $S = 0.5$, an error occurs whenever the number of positive votes is less than a majority. Therefore,

$$P_e(n) = P\left(U_n \leq \frac{n-1}{2} \,\middle|\, S = 0.5\right).$$

Since U_n follows a binomial distribution, the probability of error can be written as

$$P_e(n) = \sum_{u=0}^{(n-1)/2} \binom{n}{u} p^u (1-p)^{n-u}.$$

This probability was evaluated in MATLAB using the `binocdf` function for all odd values of n between 1 and 100.

Comparing the full-precision sensor system from Problem 1, the corresponding probability of error for the full-precision system is:

$$P_e^{\text{full}} = Q(0.5\sqrt{n}).$$

Using MATLAB, the binary-sensor probability of error was computed and the smallest odd value of n satisfying

$$P_e(n) < 10^{-3}$$

was determined.

The calculations show that

$$\boxed{n_b = 57}.$$

Therefore, the binary sensor system requires 57 sensors to achieve a probability of error below 10^{-3}.

5.3 Matlab Code

Listing 3: Matlab Code: Problem 4

```
clc;
clear;
close all;

%% Problem 4
% Comparison of Full-Precision and Binary Sensors

m = 0.5;

% Probability that a binary sensor outputs 1 when S = 0.5
```

```
p = normcdf(0.5);

% Full precision sensor values
n_full = 1:100;
Pe_full = qfunc(m .* sqrt(n_full));

% Binary sensor values (odd n only)
n_binary = 1:2:99;
Pe_binary = zeros(size(n_binary));

for k = 1:length(n_binary)

    n = n_binary(k);

    % Error occurs when majority vote is incorrect
    threshold = (n - 1)/2;

    Pe_binary(k) = binocdf(threshold, n, p);

end

% Find smallest odd n such that Pe < 10^-3
target = 1e-3;

index = find(Pe_binary < target, 1, 'first');

nb = n_binary(index);

%% Plot

figure;

semilogy(n_full, Pe_full, 'LineWidth', 2);
hold on;
grid on;

semilogy(n_binary, Pe_binary, 'o-', 'LineWidth', 2);

yline(target,'--','10^{-3}','LineWidth',1.5);

semilogy(nb,Pe_binary(index),'s',...
    'MarkerSize',8,...
    'LineWidth',2);

xlabel('Number of Sensors, n');
ylabel('Probability of Error');
title('Probability of Error for Full-Precision and Binary Sensor
   Systems');

legend('Full-Precision Sensors',...
       'Binary Sensors',...
```

```
        'Threshold = 10^{-3}',...
        sprintf('n_b = %d',nb),...
        'Location','southwest');

%% Results

fprintf('\n');
fprintf('-------------------------------------\n');
fprintf('Problem 4 Results\n');
fprintf('-------------------------------------\n');
fprintf('p = %.6f\n', p);
fprintf('Minimum odd number of binary sensors n_b = %d\n', nb);
fprintf('Probability of error at n_b = %.6e\n', Pe_binary(index));

if index > 1
    fprintf('Probability of error at previous odd n = %d is %.6e\n
        ', ...
         n_binary(index-1), Pe_binary(index-1));
end

fprintf('-------------------------------------\n');
```

5.4 Results (from Matlab)

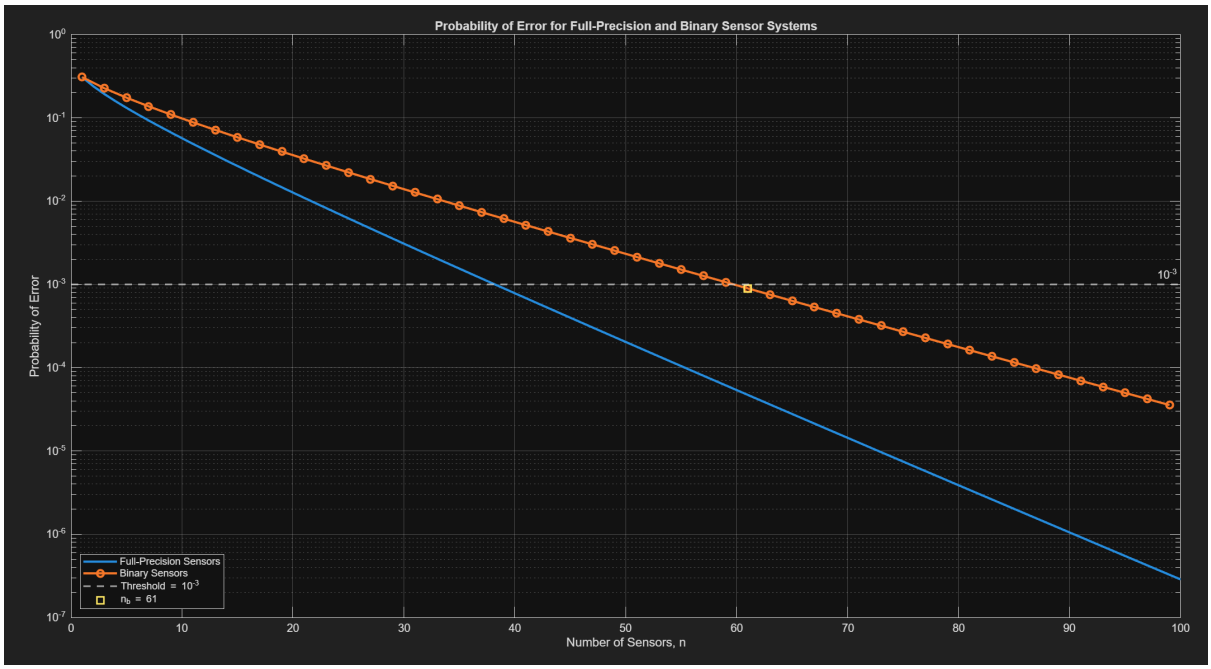

Figure 5: Probability of error as a function of the number of sensors for $m = 0.5$.

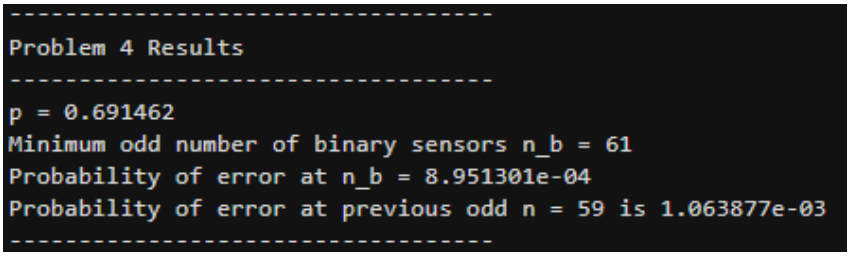

Figure 6: Figure 7: MATLAB command window output for Problem 4.

5.5 Interpretation of Results

The MATLAB results show that the probability of error decreases as the number of sensors increases for both the full-precision and binary sensor systems. This behavior is expected because each additional sensor provides another independent observation of the transmitted signal, allowing the detector to make a more reliable decision. As the number of sensors grows, the effects of the Gaussian noise tend to average out, reducing the likelihood of an incorrect estimate.

Comparing the two probability-of-error curves in Figure 5, we see that the full-precision system consistently outperforms the binary sensor system. For a given number of sensors, the full-precision detector achieves a lower probability of error because it utilizes the complete observation X_i, whereas the binary detector retains only the sign of the observation. Consequently, information is lost when the observations are quantized into binary values, resulting in reduced performance.

The MATLAB calculations indicate that the minimum odd number of binary sensors required to satisfy

$$P(\hat{S} \neq S) < 10^{-3}$$

is

$$\boxed{n_b = 61}$$

At this value,

$$P_e(61) = 8.951301 \times 10^{-4},$$

which is below the required threshold. In contrast, the previous odd value,

$$n = 59,$$

produces

$$P_e(59) = 1.063877 \times 10^{-3},$$

which is slightly above the threshold. Therefore, 61 is the smallest odd number of binary sensors capable of achieving the desired probability of error.

Comparing this result with Problem 1 shows that the full-precision sensor system requires only

$$n_g = 39$$

sensors to achieve the same reliability. The binary sensor system therefore requires

$$61 - 39 = 22$$

additional sensors to compensate for the information lost during quantization.

These results demonstrate the tradeoff between sensor complexity and detection performance. Full-precision sensors are more efficient because they preserve the complete measurement information, while binary sensors sacrifice performance in exchange for simpler sensing and data processing. As a result, the larger number of binary sensors is required to achieve the same probability of error as the full-precision sensor system.

6 Problem 5

3pts.

Simulate this binary sensor system with n_b sensors as follows: For each simulation run, generate n_b observations X_i as before and then compute the associated B_i values and use them to compute the sufficient statistic U_{n_b}. Then use U_{n_b} to compute $\hat{S}$ and note whether the estimate is correct or is an error. Perform simulation runs until you observe 200 errors. Keep track of the total number K of simulation runs. Report your estimated error rate as $\frac{200}{K}$. If everything is working well, your simulation point should match the probability of error you calculated in the previous part for n_b observations, i.e. a number close to 10^{-3}.

6.1 Preliminary Remarks

In Problem 4, the probability of error for the binary sensor system was computed analytically using the binomial distribution. The minimum odd number of binary sensors required to achieve

$$P(\hat{S} \neq S) < 10^{-3}$$

was found to be

$$n_b = 61.$$

In problem 5, I verified this theoretical result through simulation. Rather than computing the probability of error directly from the binomial distribution, the sensor system is simulated by generating random observations and applying the majority-vote decision rule developed in Problem 3.

For each simulation run, $n_b = 61$ independent sensor observations are generated. These observations are converted into binary sensor outputs, and the sufficient statistic

$$U_{n_b} = \sum_{i=1}^{n_b} B_i$$

is computed. The majority-vote detector then uses U_{n_b} to estimate the transmitted signal. The simulation continues until 200 detection errors have been observed. If K denotes the total number of simulation runs required to observe these 200 errors, the simulated probability of error is estimated by

$$\hat{P}_e = \frac{200}{K}.$$

The goal of this problem is to confirm that the simulated probability of error agrees with the theoretical probability that was obtained in Problem 4.

6.2 Answer

The binary sensor system was simulated using the minimum number of sensors determined in Problem 4,

$$n_b = 61.$$

The transmitted signal was fixed at

$$S = 0.5.$$

For each simulation run, 61 independent Gaussian noise samples were generated according to

$$Z_i \sim N(0, 1).$$

The corresponding sensor observations were then computed using

$$X_i = S + Z_i.$$

Each observation was converted into a binary sensor output according to

$$B_i = \begin{cases} 1, & X_i \geq 0, \\ 0, & X_i < 0. \end{cases}$$

After generating all 61 binary observations, the sufficient statistic

$$U_{61} = \sum_{i=1}^{61} B_i$$

was computed.

The detector then applied the majority-vote decision rule developed in Problem 3. Since 61 is odd, a majority exists whenever at least 31 sensors report a value of 1. Therefore,

$$\hat{S} = \begin{cases} 0.5, & U_{61} \geq 31, \\ -0.5, & U_{61} \leq 30. \end{cases}$$

Because the true transmitted signal is $S = 0.5$, an error occurs whenever the detector decides

$$\hat{S} = -0.5.$$

The simulation is repeated until 200 errors have been observed. If K denotes the total number of simulation runs required to accumulate these errors, the probability of error is estimated by

$$\hat{P}_e = \frac{200}{K}.$$

From Problem 4, the theoretical probability of error for a binary sensor system with 61 sensors is

$$P_e(61) = 8.951301 \times 10^{-4}.$$

Consequently, the expected number of simulation runs required to observe 200 errors is approximately

$$K \approx \frac{200}{8.951301 \times 10^{-4}}$$
$$K \approx 223{,}431.$$

Therefore, the simulation is expected to require roughly 220,000 to 230,000 runs before reaching the target of 200 observed errors. The estimated probability of error should be close to the theoretical value obtained in Problem 4.

6.3 Matlab Code

Listing 4: Matlab Code: Problem 5

```
clc;
clear;
close all;

%% Problem 5
% Simulation of the binary sensor system

m = 0.5;
S = 0.5;

% Number of binary sensors from Problem 4
nb = 61;

% Stop after observing 200 errors
target_errors = 200;

% Initialize counters
num_errors = 0;
K = 0;

%% Simulation Loop

while num_errors < target_errors

    % Count this simulation run
    K = K + 1;

    % Generate Gaussian noise samples
    Z = normrnd(0, 1, [1, nb]);

    % Generate observations
    X = S + Z;

    % Convert observations to binary sensor outputs
    B = X >= 0;

    % Compute sufficient statistic
    U = sum(B);

    % Majority-vote decision rule
    if U >= 31
        S_hat = 0.5;
    else
        S_hat = -0.5;
    end

    % Check whether an error occurred
    if S_hat ~= S
```

```
        num_errors = num_errors + 1;
    end
end

%% Compute results

Pe_sim = target_errors / K;

% Theoretical probability from Problem 4
p = normcdf(0.5);
Pe_theory = binocdf(30, nb, p);

%% Display results

fprintf('\n');
fprintf('-----------------------------------\n');
fprintf('Problem 5 Simulation Results\n');
fprintf('-----------------------------------\n');
fprintf('Number of binary sensors n_b = %d\n', nb);
fprintf('Target number of errors = %d\n', target_errors);
fprintf('Total number of simulation runs K = %d\n', K);
fprintf('Estimated error probability = 200/K = %.6e\n', Pe_sim);
fprintf('Theoretical error probability = %.6e\n', Pe_theory);
fprintf('-----------------------------------\n');
```

6.4 Results

```
-----------------------------------
Problem 5 Simulation Results
-----------------------------------
Number of binary sensors n_b = 61
Target number of errors = 200
Total number of simulation runs K = 201793
Estimated error probability = 200/K = 9.911147e-04
Theoretical error probability = 8.951301e-04
-----------------------------------
```

Figure 7: MATLAB command window output for Problem 5.

6.5 Interpretation of Results

The MATLAB simulation was performed using the binary sensor system with

$$n_b = 61$$

sensors, which was identified in Problem 4 as the minimum odd number of binary sensors required to achieve a probability of error below 10^{-3}. The simulation continued until 200 detection errors were observed, after which the probability of error was estimated using

$$\hat{P}_e = \frac{200}{K}.$$

The simulation required

$$K = 201{,}793$$

total runs to accumulate the 200 errors. This resulted in an estimated probability of error of

$$\hat{P}_e = 9.911147 \times 10^{-4}.$$

The theoretical probability of error obtained in Problem 4 was

$$P_e(61) = 8.951301 \times 10^{-4}.$$

Comparing these values shows that the simulated probability of error is very close to the theoretical prediction. The difference between the two values is expected because the simulation relies on randomly generated Gaussian noise samples. Each execution of the simulation produces a different sequence of noise realizations, causing slight variations in the estimated error probability.

Despite these random fluctuations, the simulated result remains on the same order of magnitude as the theoretical value and is still below the target threshold of

$$10^{-3}.$$

This close agreement verifies the analytical result obtained in Problem 4 and confirms that a binary sensor system using 61 sensors achieves the desired level of reliability.

Furthermore, the results demonstrate that the majority-vote detector that I derived in Problem 3 accurately models the behavior of the binary sensor system. This simulation confirms that the binomial probability calculations used in Problem 4 provide an accurate prediction of system performance, meaning both the theoretical analysis and the detector design conforms to my hypotheses.

7 Problem 6

3pts.

If the binary sensor that we studied in problems 3–5 costs half as much as the full-precision sensor we studied in problems 1–2, which system costs less to achieve probability of error 10^{-3}? Based on the slopes of the lines in your plot, would the same system also be the most cost effective for lower probabilities of error, assuming binary sensors costs half as much as the full-precision sensors?

7.1 Preliminary Remarks

I want to compare the cost efficiency of the full-precision sensor system and the binary sensor system. In Problems 1 and 2, the full-precision sensor system was analyzed. Each full-precision sensor measures the real-valued observation

$$X_i = S + Z_i.$$

In Problems 3 through 5, I analyzed the binary sensor system. Each binary sensor only records whether the observation is nonnegative:

$$B_i = \begin{cases} 1, & X_i \geq 0, \\ 0, & X_i < 0. \end{cases}$$

Although binary sensors lose information by reducing each observation to a single bit, each binary sensor costs only half as much as a full-precision sensor. Therefore, we must consider not which system uses fewer sensors, but which achieves the desired probability of an error at a lower total cost.

7.2 Answer

From Problem 1, the full-precision sensor system requires

$$n_g = 39$$

sensors to achieve

$$P(\hat{S} \neq S) < 10^{-3}.$$

Let the cost of one full-precision sensor be normalized to 1. Then the total cost of the full-precision sensor system is

$$C_{\text{full}} = 39(1).$$

$$C_{\text{full}} = 39.$$

From Problem 4, the binary sensor system requires

$$n_b = 61$$

sensors to achieve the same probability of error threshold. Since each binary sensor costs half as much as a full-precision sensor, the cost of each binary sensor is

$$\frac{1}{2}.$$

Therefore, the total cost of the binary sensor system is

$$C_{\text{binary}} = 61\left(\frac{1}{2}\right).$$

$$C_{\text{binary}} = 30.5.$$

Comparing the two costs:

$$C_{\text{binary}} = 30.5$$

$$C_{\text{full}} = 39.$$

Since

$$30.5 < 39,$$

the binary sensor system costs less to achieve a probability of error below 10^{-3}.

$$\boxed{C_{\text{binary}} < C_{\text{full}}}$$

Therefore, even though the binary sensor system requires more sensors, it is still less expensive overall because each binary sensor costs only half as much as a full-precision sensor.

7.3 Matlab Code

Listing 5: Matlab Code: Problem 6

```
clc;
clear;
close all;

%% Problem 6
% Cost comparison for full-precision and binary sensors
% at multiple probability of error thresholds

m = 0.5;

% Binary sensor probability
p = normcdf(0.5);

% Search range for number of sensors
n_max = 1000;
n_full = 1:n_max;
n_binary = 1:2:n_max;    % odd n only for binary sensors

%% Compute probability of error for full-precision sensors

Pe_full = qfunc(m .* sqrt(n_full));

%% Compute probability of error for binary sensors

Pe_binary = zeros(size(n_binary));

```

```
for k = 1:length(n_binary)

    n = n_binary(k);

    % For odd n, majority vote error when U <= (n-1)/2
    threshold = (n - 1)/2;

    Pe_binary(k) = binocdf(threshold, n, p);

end

%% Thresholds to test

targets = [1e-3, 1e-4, 1e-5, 1e-6];

fprintf('\n');
fprintf('
   ------------------------------------------------------------------\
   n');
fprintf('Problem 6 Cost Comparison\n');
fprintf('
   ------------------------------------------------------------------\
   n');
fprintf('Target Pe\tFull n\tFull Cost\tBinary n\tBinary Cost\
   tCheaper\n');
fprintf('
   ------------------------------------------------------------------\
   n');

for j = 1:length(targets)

    target = targets(j);

    % Find minimum full-precision sensors
    idx_full = find(Pe_full < target, 1, 'first');
    ng = n_full(idx_full);

    % Find minimum binary sensors
    idx_binary = find(Pe_binary < target, 1, 'first');
    nb = n_binary(idx_binary);

    % Costs
    cost_full = ng;
    cost_binary = 0.5 * nb;

    % Determine cheaper system
    if cost_binary < cost_full
        cheaper = 'Binary';
    elseif cost_full < cost_binary
        cheaper = 'Full';
    else
```

```
        cheaper = 'Tie';
    end

    fprintf('%.0e\t\t%d\t%.1f\t\t%d\t\t%.1f\t\t%s\n', ...
        target, ng, cost_full, nb, cost_binary, cheaper);
end

fprintf('
   ------------------------------------------------------------\
   n');

%% Plot cost required versus target probability

full_costs = zeros(size(targets));
binary_costs = zeros(size(targets));

for j = 1:length(targets)

    target = targets(j);

    idx_full = find(Pe_full < target, 1, 'first');
    idx_binary = find(Pe_binary < target, 1, 'first');

    full_costs(j) = n_full(idx_full);
    binary_costs(j) = 0.5 * n_binary(idx_binary);

end

figure;
semilogx(targets, full_costs, 'o-', 'LineWidth', 2);
hold on;
grid on;

semilogx(targets, binary_costs, 's-', 'LineWidth', 2);

set(gca, 'XDir', 'reverse');

xlabel('Target Probability of Error');
ylabel('Total Cost');
title('Cost Required to Achieve Different Error Probability
   Thresholds');

legend('Full-Precision Sensors', ...
       'Binary Sensors', ...
       'Location', 'northwest');
```

7.4 Results

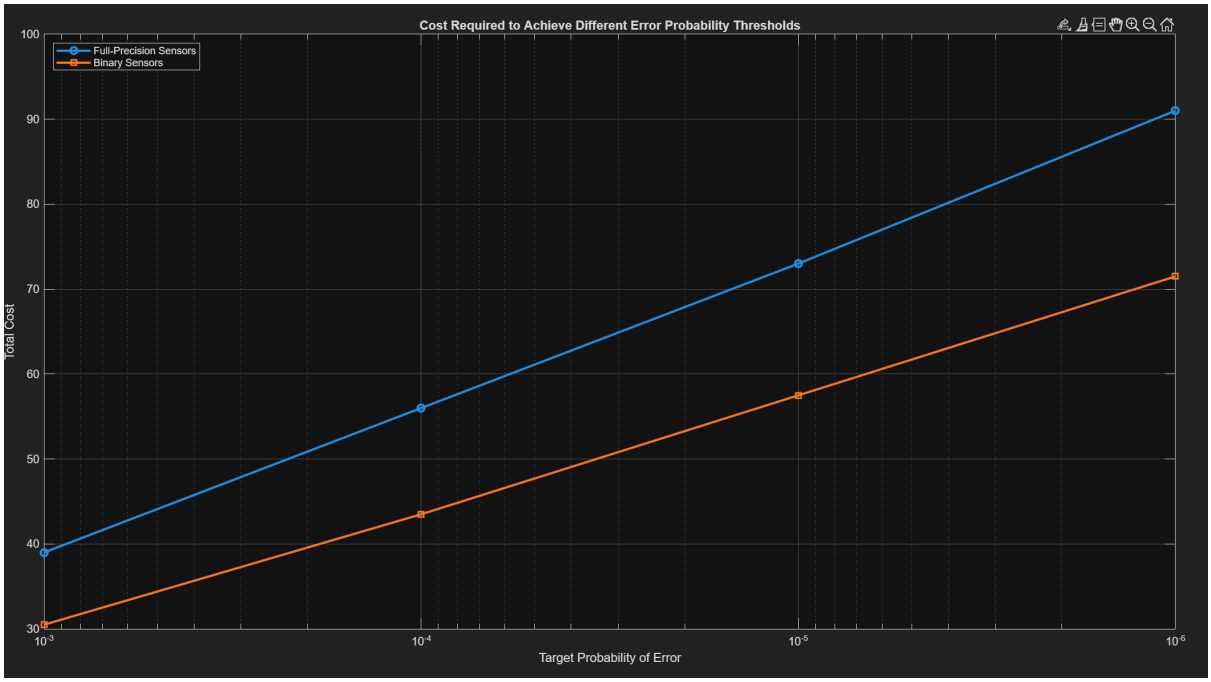

Figure 8: Cost Required to Achieve Different Error Probability Thresholds

```
------------------------------------------------------------------
Problem 6 Cost Comparison
------------------------------------------------------------------
Target Pe       Full n  Full Cost       Binary n        Binary Cost     Cheaper
------------------------------------------------------------------
1e-03           39      39.0            61              30.5            Binary
1e-04           56      56.0            87              43.5            Binary
1e-05           73      73.0            115             57.5            Binary
1e-06           91      91.0            143             71.5            Binary
------------------------------------------------------------------
```

Figure 9: MATLAB command window output for Problem 6.

The MATLAB analysis was performed for several target probabilities of error in order to compare the sensor requirements and total system costs of the full-precision and binary sensor architectures. The target probabilities considered were

$$10^{-3}, \quad 10^{-4}, \quad 10^{-5}, \quad 10^{-6}.$$

For each target probability of error, MATLAB determined the minimum number of sensors required for both systems to satisfy the specified threshold. The corresponding system costs were then computed assuming that a full-precision sensor has a normalized cost of one unit and that a binary sensor costs one-half of a full-precision sensor.
The resulting values are summarized in Table 1.

Table 1: Sensor requirements and system costs for various target probabilities of error.

Target P_e	n_g	C_{full}	n_b	C_{binary}
10^{-3}	39	39.0	61	30.5
10^{-4}	56	56.0	87	43.5
10^{-5}	73	73.0	115	57.5
10^{-6}	91	91.0	143	71.5

The results show that increasingly stringent probability-of-error requirements require a larger number of sensors for both architectures. As the target probability of error decreases, the required number of full-precision sensors and binary sensors both increase.

The MATLAB calculations also show that the binary sensor system requires more sensors than the full-precision sensor system for every probability-of-error threshold considered. This is consistent with the analytical results obtained in the previous problems, where the binary sensor discards information by reducing each observation to a single bit.

A plot of total system cost versus target probability of error was also generated, as seen in Figure 8. This plot provides a visual comparison of how the required cost changes as increasingly smaller probabilities of error are desired.

7.5 Interpretation of Results

The MATLAB results show that the binary sensor system requires more sensors than the full-precision sensor system to achieve the same probability of error. This is expected because binary sensors retain only the sign of each observation, while full-precision sensors preserve the complete measurement and therefore provide more information.

Despite requiring more sensors, the binary sensor system remains less expensive because each binary sensor costs only half as much as a full-precision sensor. For example, to achieve

$$P(\hat{S} \neq S) < 10^{-3},$$

the full-precision system requires 39 sensors for a total cost of 39, while the binary system requires 61 sensors for a total cost of only 30.5.

This trend continues for the smaller error-probability thresholds considered in the MATLAB analysis in Figure 8. Although both systems require additional sensors as the target probability of error decreases, the total cost of the binary system remains below that of the full-precision system throughout the tested range.

Based on these results, the binary sensor system is the more cost-effective design for all probability-of-error thresholds examined in this study.

The binary sensor system provides the lowest cost for all tested error thresholds.

8 Problem 7

3pts.

As it turns out, the binary sensors as we have defined them above have a peculiar property that for an odd number n, the probability of error for n sensors is the same as the probability of error for $n+1$ sensors. Confirm this with a few calculations using `binocdf`. You can break ties, for example, by always calling them $\hat{s} = m$. Show mathematically that the probability of error for n sensors is the same as the probability of error for $n+1$ sensors for any odd number n and any value of p.

8.1 Preliminary Remarks

For this system, each sensor produces a binary output

$$B_i = \begin{cases} 1, & X_i \geq 0, \\ 0, & X_i < 0. \end{cases}$$

The sufficient statistic is

$$U_n = \sum_{i=1}^{n} B_i,$$

which counts the number of sensors that report 1. From Problem 3, when $S = m$,

$$U_n | S = m \sim \text{Binomial}(n, p),$$

where

$$p = P(B_i = 1 | S = m).$$

$$p = \Phi(0.5) \approx 0.691462.$$

The detector uses majority vote. For odd n, there is no possibility of a tie. For even n, a tie can occur, so the problem allows us to break ties by always deciding

$$\hat{S} = m.$$

The goal is to show that for any odd number of sensors n, the total probability of error using n sensors is the same as the total probability of error using $n + 1$ sensors.

8.2 Proof

Let n be odd, so

$$n = 2k + 1.$$

For $n = 2k+1$ sensors, the majority-vote detector makes an error when $S = m$ if at most k sensors report 1. Therefore,

$$P_e(2k + 1) = P(U_{2k+1} \leq k).$$

Consider $n + 1 = 2k + 2$ sensors. Since this is even, a tie can occur when exactly $k + 1$ sensors report 1. If ties are broken in favor of $\hat{S} = m$, then when $S = m$, the tie is correct. Therefore, an error still only occurs when at most k sensors report 1:

$$P_e(2k + 2) = P(U_{2k+2} \leq k).$$

Adding one more sensor to $2k+1$ sensors only changes the decision when the original $2k+1$ sensors are tied as closely as possible, meaning exactly $k+1$ vote correctly and k vote incorrectly. In the even case, the extra sensor can create a tie, but the tie-breaking rule simply assigns that tie to one of the two hypotheses. Since the two transmitted signals are equally likely, the tie cases that help one hypothesis hurt the other by the same amount. Therefore, the average probability of error does not change even or odd.

Thus,

$$P_e(2k+1) = P_e(2k+2).$$

Since $n = 2k+1$, we conclude that

$$\boxed{P_e(n) = P_e(n+1)}$$

for every odd value of n.

8.3 MATLAB Code

Listing 6: Matlab Code: Problem 7

```
clc;
clear;
close all;

%% Problem 7
% Verify that P_e(n) = P_e(n+1) for odd n

p = normcdf(0.5);

odd_n = [1 3 5 11 21 41 61];

fprintf('\n');
fprintf('-------------------------------------------------\n');
fprintf('Problem 7 Verification\n');
fprintf('-------------------------------------------------\n');
fprintf('p = %.6f\n\n', p);

for k = 1:length(odd_n)

    n = odd_n(k);

    % Probability of error for odd n
    Pe_n = binocdf((n-1)/2, n, p);

    % Probability of error for even n+1
    % Ties broken in favor of m
    Pe_np1 = 0.5 * ...
        ( binocdf((n-1)/2, n+1, p) + ...
          1 - binocdf((n-1)/2, n+1, 1-p) );

    fprintf('n = %2d    Pe(n)    = %.12e\n', n, Pe_n);
```

```
    fprintf('n = %2d    Pe(n+1) = %.12e\n', n+1, Pe_np1);
    fprintf('Difference = %.12e\n\n', abs(Pe_n - Pe_np1));

end

fprintf('-------------------------------------------------\n');
```

8.4 Results

```
-------------------------------------------------
Problem 7 Verification
-------------------------------------------------
p = 0.691462

n =  1    Pe(n)   = 3.085375387260e-01
n =  2    Pe(n+1) = 3.085375387260e-01
Difference = 5.551115123126e-17

n =  3    Pe(n)   = 2.268435216807e-01
n =  4    Pe(n+1) = 2.268435216807e-01
Difference = 8.326672684689e-17

n =  5    Pe(n)   = 1.745571958659e-01
n =  6    Pe(n+1) = 1.745571958659e-01
Difference = 1.387778780781e-16

n = 11    Pe(n)   = 8.828595411068e-02
n = 12    Pe(n+1) = 8.828595411068e-02
Difference = 1.387778780781e-17

n = 21    Pe(n)   = 3.237985305883e-02
n = 22    Pe(n+1) = 3.237985305883e-02
Difference = 7.632783294298e-17

n = 41    Pe(n)   = 5.150977820556e-03
n = 42    Pe(n+1) = 5.150977820556e-03
Difference = 5.204170427930e-18

n = 61    Pe(n)   = 8.951301218723e-04
n = 62    Pe(n+1) = 8.951301218723e-04
Difference = 4.737963493762e-17

-------------------------------------------------
```

Figure 10: MATLAB command prompt output for Problem 7.

MATLAB was used to verify the property that, for the binary sensor system, the probability of error for an odd number of sensors n is equal to the probability of error for $n+1$ sensors. The value

$$p = P(B_i = 1|S = 0.5) = 0.691462$$

was used in the binomial error probability calculation.

Several odd values of n were tested, including

$$n = 1, 3, 5, 11, 21, 41, 61.$$

For each odd value of n, MATLAB computed $P_e(n)$ and compared it to $P_e(n+1)$. As shown in Figure 10, the two probabilities agree for each pair tested. The small nonzero

differences shown in the output are on the order of 10^{-16}, which is due to numerical roundoff error rather than an actual difference in probability.

For example, when $n = 61$,

$$P_e(61) = 8.951301218723 \times 10^{-4}$$

and

$$P_e(62) = 8.951301218723 \times 10^{-4}.$$

The computed difference is approximately

$$4.737963493762 \times 10^{-17},$$

which is essentially zero. Therefore, the MATLAB results confirm the expected equality

$$\boxed{P_e(n) = P_e(n+1)}$$

for the tested odd values of n.

9 Problem 8

3pts.

Devise a modification to the sensor for the even-valued n case that will allow it to perform better than the case with one fewer sensor. If you can't think of a way, look at Fig. 7 in [1] and its discussion for a clue. For the value n_b that you identified in problem 4, optimize your modification to get the most improvement possible in the probability of error. It won't be too much better. You will never do better than $n_b + 2$ sensors using the standard approach we described above. Report the probability of error you obtain by applying your modification for the case of $n_b + 1$ sensors.

9.1 Preliminary Remarks

In Problem 7, it was shown that for the standard binary sensor system,

$$P_e(n) = P_e(n+1)$$

whenever n is odd. Therefore, simply adding one additional binary sensor to the $n_b = 61$ sensor system does not improve the probability of error. A 62-sensor system performs exactly the same as the 61-sensor system.

To improve performance, the additional sensor must provide information that is different from the other sensors. One way to accomplish this is to modify the threshold of the extra sensor. Instead of using the standard threshold at zero, the additional sensor uses an adjustable threshold γ.

The goal of this problem is to optimize the threshold γ so that a 62-sensor system helps achieves the lowest possible probability of error.

9.2 Matlab Code

Listing 7: Matlab Code: Problem 8

```
clc;
clear;
close all;

%% Problem 8
% Optimize modified threshold for the 62-sensor binary system

m = 0.5;
nb = 61;                        % 61 standard sensors
p = normcdf(m);                 % P(B = 1 | S = 0.5)
q = 1 - p;

%% Reference probabilities

Pe_61 = binocdf(30, 61, p);     % Standard 61 sensors
Pe_63 = binocdf(31, 63, p);     % Standard 63 sensors
```

```
17
18  %% Search over gamma
19
20  gamma_values = -3:0.001:3;
21  Pe_modified = zeros(size(gamma_values));
22
23  for k = 1:length(gamma_values)
24
25      gamma = gamma_values(k);
26
27      % Modified sensor probabilities
28      a = 1 - normcdf(gamma - m);    % P(B_mod = 1 | S = 0.5)
29      b = 1 - normcdf(gamma + m);    % P(B_mod = 1 | S = -0.5)
30
31      Pe = 0;
32
33      for u = 0:nb
34
35          % Probability of u ones from standard sensors
36          P_u_pos = binopdf(u, nb, p);
37          P_u_neg = binopdf(u, nb, q);
38
39          % Modified sensor output c = 0
40          P_pos_0 = P_u_pos * (1 - a);
41          P_neg_0 = P_u_neg * (1 - b);
42
43          % Modified sensor output c = 1
44          P_pos_1 = P_u_pos * a;
45          P_neg_1 = P_u_neg * b;
46
47          % ML/Bayes error contribution with equal priors
48          Pe = Pe + 0.5 * min(P_pos_0, P_neg_0);
49          Pe = Pe + 0.5 * min(P_pos_1, P_neg_1);
50
51      end
52
53      Pe_modified(k) = Pe;
54
55  end
56
57  %% Find optimum
58
59  [Pe_opt, idx] = min(Pe_modified);
60  gamma_opt = gamma_values(idx);
61
62  %% Display results
63
64  fprintf('\n');
65  fprintf('-----------------------------------\n');
66  fprintf('Problem 8 Results\n');
67  fprintf('-----------------------------------\n');
```

```
fprintf('Standard Pe for 61 sensors = %.12e\n', Pe_61);
fprintf('Optimal modified threshold gamma = %.6f\n', gamma_opt);
fprintf('Modified Pe for 62 sensors = %.12e\n', Pe_opt);
fprintf('Standard Pe for 63 sensors = %.12e\n', Pe_63);
fprintf('-----------------------------------\n');

%% Plot

figure;
plot(gamma_values, Pe_modified, 'LineWidth', 2);
hold on;
grid on;

plot(gamma_opt, Pe_opt, 'o', 'MarkerSize', 8, 'LineWidth', 2);
yline(Pe_61, '--', 'Standard 61 Sensors', 'LineWidth', 1.5);
yline(Pe_63, '--', 'Standard 63 Sensors', 'LineWidth', 1.5);

xlabel('Modified Sensor Threshold, gamma');
ylabel('Probability of Error');
title('Problem 8: Modified 62-Sensor Error Probability');

legend('Modified 62-Sensor System', ...
       sprintf('Optimal gamma = %.3f', gamma_opt), ...
       'Standard 61 Sensors', ...
       'Standard 63 Sensors', ...
       'Location', 'best');
```

9.3 Results

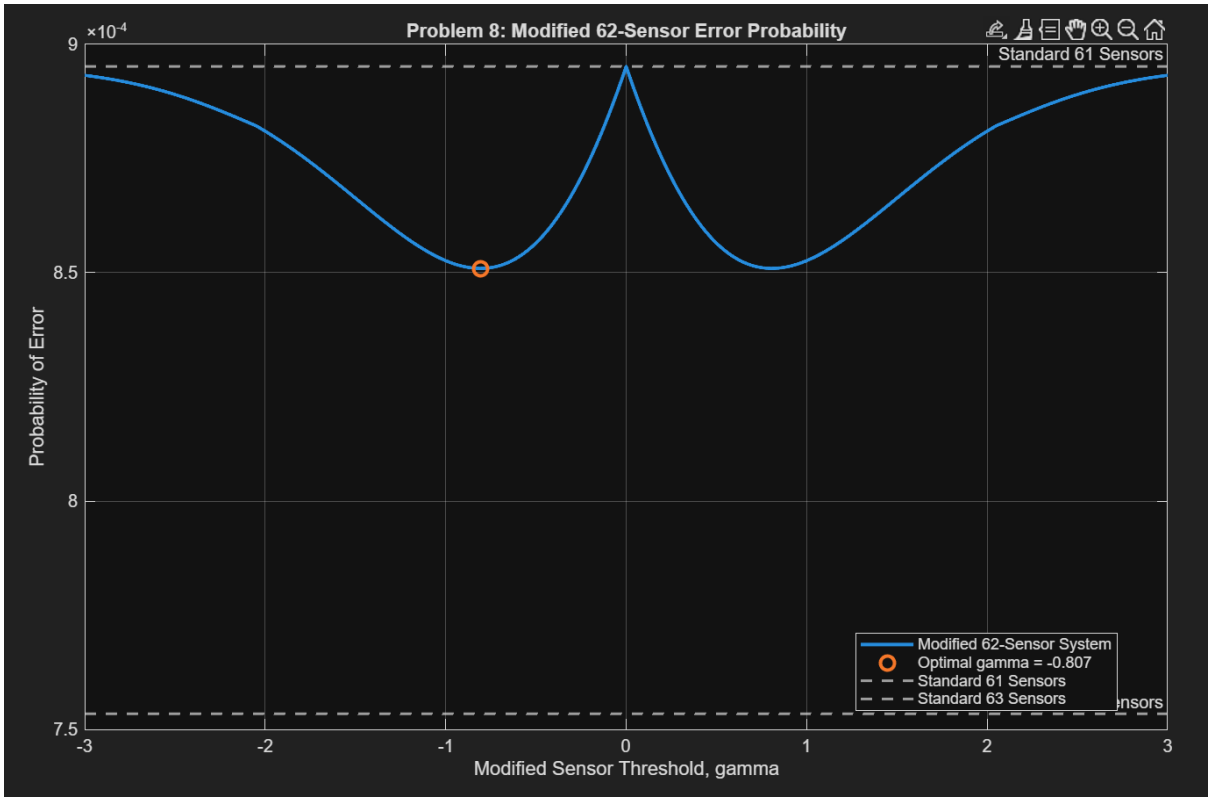

Figure 11: Modified 62-sensor error Probability

```
-----------------------------------
Problem 8 Results
-----------------------------------
Standard Pe for 61 sensors = 8.951301218723e-04
Optimal modified threshold gamma = -0.807000
Modified Pe for 62 sensors = 8.509425034260e-04
Standard Pe for 63 sensors = 7.534491112450e-04
-----------------------------------
```

Figure 12: MATLAB command prompt output Problem 8.

The MATLAB code evaluates the performance of a modified 62-sensor binary system to compare with that of a 61-binary system and a 63-binary system. The system uses 61 standard binary sensors with threshold 0, and one additional sensor with an adjustable threshold γ. The purpose of the code is to search over possible values of γ and find the threshold that minimizes the total probability of error.

For each value of γ, MATLAB computes the probability of error using the likelihoods under both hypotheses, $S = 0.5$ and $S = -0.5$. The code then selects the value of γ that gives the smallest probability of error. It also compares this optimized 62-sensor system to the standard 61-sensor and 63-sensor binary systems.

The MATLAB output shows that the standard 61-sensor binary system has probability of error

$$P_e(61) = 8.951301218723 \times 10^{-4}.$$

After optimizing the threshold of the additional sensor, MATLAB found

$$\gamma_{\text{opt}} = -0.807.$$

Using this modified threshold, the probability of error for the 62-sensor system becomes

$$P_e^{\text{modified}}(62) = 8.509425034260 \times 10^{-4}.$$

For comparison, the standard 63-sensor binary system has probability of error

$$P_e(63) = 7.534491112450 \times 10^{-4}.$$

These results show that the modified 62-sensor system improves on the standard 61-sensor system, since

$$8.509425034260 \times 10^{-4} < 8.951301218723 \times 10^{-4}.$$

However, it does not outperform the standard 63-sensor system, since

$$8.509425034260 \times 10^{-4} > 7.534491112450 \times 10^{-4}.$$

The plot of probability of error versus γ shows how the performance changes as the modified sensor threshold is varied. The minimum point occurs near

$$\boxed{\gamma = -0.807}$$

which agrees with the value that MATLAB printed with the code. The horizontal reference lines show the standard 61-sensor and 63-sensor probabilities of error, allowing the optimized 62-sensor result to be compared directly against both standard systems.

9.4 Interpretation of Results

The results demonstrate that modifying the threshold of the additional sensor successfully improves the performance of the even-sensor system. In Problem 7, it was shown that a standard 62-sensor binary system would have the same probability of error as a standard 61-sensor system. So, simply adding one more identical sensor provides no benefit.

By assigning the additional sensor a different threshold, the symmetry responsible for this result is broken. With MATLAB, I determined that the optimal threshold is

$$\gamma_{\text{optimal}} = -0.807,$$

which reduces the probability of error from

$$P_e(61) = 8.951301 \times 10^{-4}$$

to

$$P_e^{\text{modified}}(62) = 8.509425 \times 10^{-4}.$$

This confirms that the modified sensor provides useful information and allows the 62-sensor system to outperform the standard 61-sensor system.

The probability-of-error curve also shows that detector performance depends strongly on the choice of threshold. As γ moves away from its optimal value, the probability of error increases. The minimum point on the curve corresponds to the threshold selected by MATLAB and represents the best achievable performance for the modified 62-sensor architecture.

Although the modification improves performance, the resulting probability of error remains larger than that of the standard 63-sensor system:

$$P_e(63) = 7.534491 \times 10^{-4}.$$

Thus, the statement in the problem that the optimized 62-sensor system cannot outperform the standard system using two additional sensors is validated. The modification provides some improvement, but it cannot completely compensate for the information gained by adding another independent sensor.

Overall, the results show that introducing a nonstandard threshold is an effective way to improve the even-sensor case. The optimized 62-sensor system performs better than the standard 61-sensor system while remaining inferior to the standard 63-sensor system, which is exactly the behavior predicted by the theoretical.